# 1. Czym jest Git?

Git to system kontroli wersji. Pozwala zapisywać kolejne wersje projektu, porównywać zmiany, wracać do wcześniejszych wersji i pracować nad kilkoma wariantami programu jednocześnie.

Najważniejsze pojęcia:

- **Repozytorium lokalne** — projekt wraz z jego historią zmian na Twoim komputerze.
- **Commit** — zapisany punkt w historii projektu, zawierający wybrane zmiany.
- **Branch (gałąź)** — niezależny ciąg rozwoju projektu, np. do testowania nowej funkcji.
- **Remote (repozytorium zdalne)** — repozytorium znajdujące się na serwerze.
- **Push** — wysłanie lokalnych commitów na serwer.
- **Fetch** — pobranie informacji o zmianach na serwerze bez integrowania ich z aktualną gałęzią.
- **Pull** — pobranie zmian i próba zintegrowania ich z aktualną gałęzią.
- **Merge** — połączenie zmian z dwóch gałęzi.

Typowy przebieg pracy wygląda tak:

1. Edytujesz pliki projektu.
2. Wybierasz zmiany, które mają wejść do commitu.
3. Tworzysz commit.
4. Wysyłasz commit na serwer.
5. Na innym komputerze pobierasz zmiany i kontynuujesz pracę.

# 2. Instalacja Gita

## Debian / Ubuntu

```
sudo apt update
sudo apt install git
```

Sprawdź, czy Git działa:

```
git --version
```

Git powinien wyświetlić numer zainstalowanej wersji.

# 3. Pierwsza konfiguracja

Konfigurację wykonujesz zazwyczaj tylko raz dla swojego konta użytkownika.

## 3.1. Nazwa i adres e-mail

```
git config --global user.name "Twoje Imię i Nazwisko"
git config --global user.email "twoj-email@example.com"
```

Git zapisuje te dane w każdym nowym commicie jako autora zmiany.

Opcja `--global` oznacza, że konfiguracja dotyczy wszystkich repozytoriów danego użytkownika, chyba że ustawisz inne wartości dla konkretnego projektu.

Sprawdź ustawienia:

```
git config --global user.name
git config --global user.email
```

## 3.2. Domyślna nazwa gałęzi

Ustaw `main` jako domyślną nazwę początkowej gałęzi:

```
git config --global init.defaultBranch main
```

Dzięki temu nowe repozytoria będą rozpoczynać historię od gałęzi `main`.

## 3.3. Edytor tekstu

Jeśli korzystasz z Linuksa i edytora gedit, możesz ustawić go jako edytor uruchamiany przez Gita:

```
git config --global core.editor "gedit --wait"
```

Opcja `--wait` sprawia, że Git czeka na zamknięcie edytora, zanim przejdzie dalej. To przydatne m.in. podczas edycji komunikatów commitów.

## 3.4. Wyświetlenie całej konfiguracji

```
git config --list --show-origin
```

Opcja `--show-origin` pokazuje również plik, z którego pochodzi dane ustawienie.

Konfigurację globalną znajdziesz zwykle w:

```
~/.gitconfig
```

Konfiguracja konkretnego repozytorium jest zapisywana w jego katalogu:

`.git/config`

# 4. Utworzenie nowego projektu

Załóżmy, że chcemy utworzyć projekt o nazwie `moj-projekt`.

## 4.1. Utworzenie katalogu

```
mkdir -p ~/projekty/moj-projekt
cd ~/projekty/moj-projekt
```

Katalog zostanie utworzony w `~/projekty`, czyli w podkatalogu Twojego katalogu domowego.

## 4.2. Inicjalizacja repozytorium

```
git init
```

Git utworzy ukryty katalog `.git`, w którym przechowuje historię i konfigurację repozytorium.

Sprawdź stan projektu:

```
git status
```

Powinien pojawić się komunikat informujący o gałęzi `main` oraz braku commitów.

Od tego momentu katalog projektu jest repozytorium Git.

## 4.3. Utworzenie plików

Stwórz plik `README.md`:

```
cat > README.md <<'EOF'
# Moj projekt

Opis projektu.

## Uruchamianie

Instrukcja uruchamiania programu.
EOF
```

Utwórz przykładowy program:

```
echo 'print("Witaj, świecie!")' > main.py
```

Dodaj plik `.gitignore`:

```
cat > .gitignore <<'EOF'
.env
*.log
__pycache__/
*.pyc
build/
dist/
EOF
```

Plik `.gitignore` określa, których nieśledzonych plików Git nie powinien dodawać do repozytorium. W przykładzie pomijamy m.in. pliki logów, pliki tymczasowe Pythona i lokalny plik `.env`.

Sprawdź stan repozytorium:

```
git status
```

Git pokaże nowe pliki, które nie zostały jeszcze dodane do historii.

# 5. Pierwszy commit

Git rozdziela pracę na trzy obszary:

1. **Working tree** — pliki, które edytujesz.
2. **Staging area (index)** — zmiany przygotowane do zapisania.
3. **Historia commitów** — zmiany już zapisane w repozytorium.

## 5.1. Przygotowanie plików

Dodaj pliki do staging area:

```
git add README.md main.py .gitignore
```

Można też dodać wszystkie zmiany w bieżącym katalogu:

```
git add .
```

Przed użyciem `git add .` warto sprawdzić, czy nie dodajesz przypadkiem haseł, kluczy prywatnych lub dużych plików, których nie chcesz przechowywać w repozytorium.

Sprawdź stan:

```
git status
```

Teraz pliki powinny być oznaczone jako przygotowane do commitu.

Możesz zobaczyć dokładne zmiany, które zostaną zapisane:

```
git diff --cached
```

## 5.2. Zapisanie commitu

```
git commit -m "Utworzenie początkowej struktury projektu"
```

To Twój pierwszy commit.

Parametr `-m` określa komunikat opisujący zmianę. Dobrze, aby był krótki i konkretny, np.:

```
git commit -m "Dodanie obsługi konfiguracji"
git commit -m "Naprawa błędu odczytu pliku"
git commit -m "Aktualizacja dokumentacji"
```

Commit zapisuje zmiany w lokalnym repozytorium. **Nie wysyła ich na serwer.**

## 5.3. Podgląd historii

```
git log
```

Skrócona historia:

```
git log --oneline
```

Bardziej rozbudowany podgląd, uwzględniający gałęzie:

```
git log --oneline --decorate --graph --all
```

Każdy commit ma swój identyfikator, zwykle przedstawiany jako ciąg znaków szesnastkowych.

## 5.4. Kolejna zmiana

Zmień plik:

```
echo 'print("Druga wersja programu")' >> main.py
```

Sprawdź różnice:

```
git diff
```

W tym momencie zmiana nie jest jeszcze przygotowana do commitu.

Dodaj ją i zapisz:

```
git add main.py
git diff --cached
git commit -m "Dodanie drugiego komunikatu"
```

Masz teraz dwa commity w historii.

# 6. Praca z gałęziami — branch

Gałęzie pozwalają rozwijać funkcje niezależnie od głównej wersji projektu.

Przykład: pracujesz nad gałęzią `main`, ale chcesz dodać nową funkcję. Tworzysz do tego osobną gałąź.

## 6.1. Wyświetlenie gałęzi

```
git branch
```

Gwiazdką oznaczona jest aktualnie wybrana gałąź.

## 6.2. Utworzenie nowej gałęzi i przejście do niej

```
git switch -c feature/opis
```

Polecenie tworzy gałąź `feature/opis` i od razu przełącza na nią repozytorium.

Możesz sprawdzić wynik:

```
git branch
```

## 6.3. Praca na nowej gałęzi

Edytuj plik:

```
echo "Dodano nową funkcję." >> README.md
```

Zapisz zmianę jako commit:

```
git add README.md
git commit -m "Dodanie opisu nowej funkcji"
```

Commit trafia na gałąź `feature/opis`, a nie bezpośrednio na `main`.

## 6.4. Powrót do main

```
git switch main
```

Możesz sprawdzić historię obu gałęzi:

```
git log --oneline --decorate --graph --all
```

Zauważysz, że commit utworzony na `feature/opis` nie należy jeszcze do historii `main`.

### 6.5. Połączenie gałęzi

Będąc na `main`, wykonaj:

```
git merge feature/opis
```

Git połączy zmiany z gałęzi funkcjonalnej z aktualną gałęzią. Jeżeli historia na to pozwala, może wykonać tzw. fast-forward, czyli przesunąć wskaźnik `main` do nowszego commitu bez tworzenia dodatkowego commitu scalającego.

Sprawdź wynik:

```
git log --oneline --decorate --graph --all
```

Po udanym scaleniu gałąź funkcjonalną można usunąć:

```
git branch -d feature/opis
```

Usunięcie gałęzi nie usuwa commitów, które zostały już scalone z `main`.

### 6.6. Najważniejsze polecenia dotyczące gałęzi

```
git branch
git switch main
git switch -c feature/nazwa
git merge feature/nazwa
git branch -d feature/nazwa
```

## 7. Wysłanie projektu na GitHub lub GitLab

Repozytorium lokalne nie wymaga serwera. Możesz jednak przechowywać kopię zdalną, aby zabezpieczyć historię i synchronizować projekt między komputerami.

W przykładzie używamy adresu SSH. Podobny schemat działa w GitHubie, GitLabie i na wielu innych serwerach Git.

### 7.1. Utworzenie pustego repozytorium na serwerze

Zaloguj się do wybranej platformy i utwórz nowe repozytorium, np. `moj-projekt`.

Jeżeli projekt istnieje już lokalnie, na początek najłatwiej utworzyć repozytorium zdalne bez automatycznego dodawania pliku README, licencji ani `.gitignore`. Dzięki temu obie strony nie będą miały niezależnych historii początkowych.

## 7.2. Uwierzytelnienie przez SSH

Jeżeli nie masz jeszcze klucza SSH, utwórz go na swoim komputerze:

```
ssh-keygen -t ed25519 -C "twoj-email@example.com"
```

Możesz zaakceptować domyślną ścieżkę zapisu. Ustawienie hasła do klucza jest zalecane.

Wyświetl klucz publiczny:

```
cat ~/.ssh/id_ed25519.pub
```

Skopiuj wyświetloną zawartość do ustawień kluczy SSH swojego konta na wybranej platformie.

**Nie udostępniaj pliku `~/.ssh/id_ed25519`** — to klucz prywatny. Do konfiguracji konta służy plik z końcówką `.pub`.

Dla GitHuba możesz przetestować połączenie:

```
ssh -T git@github.com
```

Dla GitLaba użyj odpowiedniego adresu jego serwera.

## 7.3. Powiązanie lokalnego repozytorium z serwerem

W katalogu projektu wykonaj:

```
git remote add origin git@github.com:NAZWA_UZYTKOWNIKA/moj-projekt.git
```

Zastąp nazwę użytkownika i adres przykładowego repozytorium rzeczywistymi danymi.

`origin` jest umowną, powszechnie stosowaną nazwą zdalnego repozytorium. Można użyć innej nazwy, ale `origin` jest dobrym standardem.

Sprawdź konfigurację:

```
git remote -v
```

Powinien pojawić się adres zdalnego repozytorium dla pobierania i wysyłania.

### 7.4. Pierwsze wysłanie projektu

```
git push -u origin main
```

To polecenie wysyła commity z lokalnej gałęzi `main` do repozytorium `origin`.

Opcja `-u` ustawia gałąź zdalną jako gałąź śledzoną (upstream). W kolejnych operacjach na tej gałęzi zazwyczaj wystarczy:

```
git push
```

Pierwsze wysłanie może wymagać uwierzytelnienia lub zaakceptowania połączenia SSH.

Po wykonaniu polecenia odśwież stronę repozytorium na serwerze — powinieneś zobaczyć pliki projektu oraz historię commitów.

### 7.5. Wysłanie nowej gałęzi

Jeśli utworzyłeś lokalną gałąź `feature/opis` i chcesz udostępnić ją na serwerze:

```
git switch feature/opis
git push -u origin feature/opis
```

Sama obecność gałęzi w lokalnym repozytorium nie oznacza, że została ona wysłana na serwer.

## 8. Własny serwer Git przez SSH

Nie musisz korzystać z GitHuba ani GitLaba. Możesz użyć własnego serwera z Linuksem i dostępem SSH.

W tym wariancie utworzymy repozytorium typu **bare**, przeznaczone do przechowywania historii i obsługi operacji Git, a nie do bezpośredniej edycji plików.

### 8.1. Utworzenie repozytorium na serwerze

Połącz się z serwerem:

```
ssh gituser@serwer.example
```

Zalogowany jako `gituser`, wykonaj:

```
mkdir -p ~/repos
git init --bare ~/repos/moj-projekt.git
```

Następnie zakończ sesję:

```
exit
```

Założenia przykładu:

- użytkownik na serwerze: `gituser`;
- nazwa hosta: `serwer.example`;
- repozytorium: `~/repos/moj-projekt.git`.

W praktyce podstaw prawdziwą nazwę hosta, konto i ścieżkę do repozytorium.

### 8.2. Podłączenie lokalnego projektu

Na swoim komputerze, w katalogu projektu, wykonaj:

```
git remote add origin ssh://gituser@serwer.example/home/gituser/repos/mo
```

Ten adres zakłada, że katalog domowy użytkownika `gituser` znajduje się w `/home/gituser`.

Sprawdź połączenie:

```
git remote -v
```

Wyślij projekt:

```
git push -u origin main
```

### 8.3. Klonowanie projektu na innym komputerze

Na drugim komputerze wykonaj:

```
git clone ssh://gituser@serwer.example/home/gituser/repos/moj-projekt.gi
```

Przejdź do utworzonego katalogu:

```
cd moj-projekt
```

Klonowanie pobiera pliki projektu i jego historię oraz automatycznie konfiguruje zdalne repozytorium pod nazwą `origin`.

Jeśli zamierzasz pracować na drugim komputerze nad tym samym projektem, zazwyczaj nie wykonujesz ponownie `git init` ani `git remote add origin`. Wszystko zostało już przygotowane przez `git clone`.

## 9. Synchronizacja z serwerem

To jedna z najważniejszych części pracy z Gitem.

Załóżmy, że pracujesz na dwóch komputerach. Na pierwszym tworzysz commit i wysyłasz go na serwer. Na drugim chcesz pobrać tę zmianę.

## 9.1. Pobranie zmian i aktualizacja lokalnej gałęzi

Najpierw przejdź na `main`:

```
git switch main
```

Sprawdź, czy masz niezapisane zmiany:

```
git status
```

Jeśli masz zmiany, które chcesz zachować, zapisz je w commicie albo świadomie odłóż przed aktualizacją.

Następnie pobierz i zintegruj zmiany z serwera:

```
git pull --ff-only origin main
```

Opcja `--ff-only` pozwala na aktualizację tylko wtedy, gdy można ją wykonać przez fast-forward. Dzięki temu Git nie tworzy automatycznie dodatkowego commitu scalającego, jeśli historia lokalna i zdalna się rozeszły.

Jeżeli pracujesz na innej gałęzi, odpowiednio zmień nazwę `main`.

## 9.2. Zwykły push po wcześniejszym skonfigurowaniu upstream

Jeśli jesteś na gałęzi śledzącej odpowiednią gałąź zdalną, wystarczy:

```
git push
```

Analogicznie, aby pobrać zmiany:

```
git pull
```

Warto jednak pamiętać, że `git pull` działa na aktualnie wybranej gałęzi i integruje zmiany z jej gałęzią śledzoną. Nie jest to polecenie do aktualizacji wszystkich gałęzi jednocześnie.

## 9.3. Różnica pomiędzy fetch i pull

`git fetch` pobiera informacje o serwerze i aktualizuje lokalne referencje do zdalnych gałęzi. Nie zmienia automatycznie plików aktualnej gałęzi.

```
git fetch origin
```

Po pobraniu możesz obejrzeć historię:

```
git log --oneline --graph --decorate --all
```

Jeżeli chcesz zintegrować pobraną gałąź `main`, wykonaj będąc na lokalnej gałęzi `main`:

```
git merge origin/main
```

`git pull` jest w uproszczeniu operacją pobrania zmian, a następnie ich integracji. Zachowaj rozróżnienie:

- `fetch` — pobierz informacje i commity;
- `merge` — połącz historię gałęzi;
- `pull` — pobierz i zintegruj zmiany;
- `push` — wyślij lokalne commity na serwer.

# 10. Typowy cykl pracy na dwóch komputerach

Poniższy przykład pokazuje zalecaną kolejność działań przy pracy nad projektem.

## Komputer A — praca nad projektem

Zaktualizuj lokalną gałąź główną:

```
git switch main
git pull --ff-only origin main
```

Utwórz nową gałąź:

```
git switch -c feature/nowa-funkcja
```

Edytuj pliki, przygotuj zmiany i utwórz commit:

```
git status
git add main.py
git diff --cached
git commit -m "Dodanie nowej funkcji"
```

Wyślij gałąź na serwer:

```
git push -u origin feature/nowa-funkcja
```

Możesz następnie scalić ją przez mechanizm Merge Request / Pull Request na platformie albo lokalnie.

Przykładowe scalanie lokalne:

```
git switch main
git pull --ff-only origin main
git merge feature/nowa-funkcja
git push origin main
```

Jeśli gałąź została już scalona, możesz ją usunąć:

```
git branch -d feature/nowa-funkcja
```

Jeśli wcześniej wysłałeś ją na serwer i chcesz ją tam również usunąć:

```
git push origin --delete feature/nowa-funkcja
```

Przed usuwaniem gałęzi sprawdź, czy jej zmiany rzeczywiście zostały scalone i nie są już potrzebne.

### Komputer B — pobranie pracy z komputera A

Przejdź na główną gałąź i pobierz nowe commity:

```
git switch main
git pull --ff-only origin main
```

W tym momencie lokalny `main` będzie zawierał zmiany wysłane na serwer i scalone z główną gałęzią.

Jeżeli gałąź funkcjonalna została wysłana, ale nie scalona, możesz ją pobrać i przełączyć się na nią:

```
git fetch origin
git switch --track origin/feature/nowa-funkcja
```

To utworzy lokalną gałąź śledzącą zdalną gałąź o podanej nazwie.

## 11. Konflikty podczas synchronizacji

Konflikt może wystąpić wtedy, gdy różne zmiany dotyczą tego samego fragmentu pliku i Git nie potrafi sam zdecydować, jak je połączyć.

Przyjmijmy, że na serwerze pojawiły się nowe commity, a Ty także utworzyłeś lokalne commity na `main`. Lokalna i zdalna historia mogą się wtedy rozchodzić.

Jeżeli:

```
git pull --ff-only origin main
```

odmówi aktualizacji, sprawdź sytuację:

```
git status
git fetch origin
git log --oneline --graph --decorate --all
```

Jeśli chcesz połączyć obie historie przez merge, upewnij się, że jesteś na `main`, i wykonaj:

```
git switch main
git merge origin/main
```

Jeśli wystąpi konflikt, Git oznaczy problematyczne pliki. Sprawdź je:

```
git status
```

W pliku może pojawić się fragment podobny do tego:

```
<<<<<<< HEAD
Wersja lokalna
=======
Wersja z serwera
>>>>>>> origin/main
```

Otwórz plik w edytorze, wybierz właściwą treść albo połącz obie wersje i usuń znaczniki konfliktu.

Następnie przygotuj naprawiony plik i dokończ scalanie:

```
git add README.md
git commit
```

Jeżeli konflikt dotyczy innego pliku, podaj jego nazwę zamiast `README.md`. W przypadku trwającego scalania można również użyć `git merge --continue`.

Po udanym scaleniu wyślij wynik na serwer:

```
git push origin main
```

Nie rozwiązuj problemów z synchronizacją przez przypadkowe użycie `git push --force`. Współdzielona historia mogłaby zostać nadpisana i utrudnić innym odzyskanie pracy.

# 12. Przydatne komendy na co dzień

## Stan repozytorium

```
git status
```

Pokazuje zmienione pliki, pliki przygotowane do commitu i bieżącą gałąź.

## Różnice w plikach

```
git diff
```

Pokazuje zmiany, które nie zostały jeszcze dodane do staging area.

```
git diff --cached
```

Pokazuje zmiany przygotowane do najbliższego commitu.

### Historia

```
git log --oneline
git log --oneline --graph --decorate --all
```

### Gałęzie lokalne i zdalne

```
git branch
git branch -a
git branch -vv
```

### Konfiguracja repozytoriów zdalnych

```
git remote -v
git remote show origin
```

### Pobranie informacji z serwera

```
git fetch origin
```

### Wysłanie commitów

```
git push
```

### Zapis zmian

```
git add plik.txt
git commit -m "Opis zmian"
```

## 13. Cofanie zmian — ostrożnie

Nie wszystkie zmiany trzeba od razu zapisywać w commicie. Git pozwala również wycofać niektóre operacje.

### 13.1. Usunięcie pliku ze staging area

Jeżeli omyłkowo przygotowałeś plik do commitu:

```
git restore --staged plik.txt
```

Plik pozostanie zmodyfikowany, ale przestanie być przygotowany do commitu.

### 13.2. Odrzucenie lokalnych zmian w pliku

```
git restore -- plik.txt
```

To odrzuca niezapisane w commicie zmiany w pliku i przywraca jego zawartość z aktualnej wersji zapisanej przez Git. Używaj tej komendy tylko wtedy, gdy na pewno nie potrzebujesz tych zmian.

### 13.3. Wycofanie zmiany zapisanej już w commicie

Jeżeli commit został już wysłany na serwer i chcesz odwrócić jego efekt, bezpiecznym rozwiązaniem w typowej współdzielonej historii jest:

```
git revert IDENTYFIKATOR_COMMITU
```

Git utworzy nowy commit odwracający wskazane zmiany. Zastąp `IDENTYFIKATOR_COMMITU` rzeczywistym identyfikatorem z `git log --oneline`.

Nie należy traktować `git restore`, `git revert` i `git reset` jako zamienników — operują na różnych etapach pracy z historią.

## 14. Najczęstsze problemy

### „Author identity unknown”

Git nie ma ustawionej nazwy lub adresu e-mail autora.

Rozwiązanie:

```
git config --global user.name "Twoje Imię i Nazwisko"
git config --global user.email "twoj-email@example.com"
```

### „remote origin already exists”

Zdalne repozytorium `origin` jest już skonfigurowane.

Sprawdź jego adres:

```
git remote -v
```

Jeśli jest nieprawidłowy, możesz go zmienić:

```
git remote set-url origin git@github.com:NAZWA_UZYTKOWNIKA/moj-projekt.g
```

### „Permission denied (publickey)”

Najczęściej oznacza to problem z uwierzytelnieniem SSH. Sprawdź, czy klucz publiczny został dodany do właściwego konta na serwerze, czy używasz odpowiedniego adresu i czy konto ma dostęp do repozytorium.

### „Updates were rejected”

Serwer ma commity, których nie ma w Twojej lokalnej gałęzi, więc Git nie chce zaakceptować wysyłania, które mogłoby nadpisać historię.

Zacznij od pobrania informacji:

```
git fetch origin
git log --oneline --graph --decorate --all
```

Sprawdź historię i zintegruj zmiany świadomie. Jeżeli obie strony mają niezależne commity, może być potrzebny merge albo rebase.

### Przypadkowo dodałem plik z hasłem

Nie zapisuj haseł, tokenów, kluczy prywatnych ani zawartości `.env` w repozytorium.

Dodanie nazwy pliku do `.gitignore` nie usuwa go z historii, jeśli był już śledzony. Aby przestać śledzić plik bez usuwania go z dysku, można użyć:

```
git rm --cached .env
```

Następnie zapisz tę zmianę w commicie. Jeżeli sekret został już wysłany na serwer, samo usunięcie go w kolejnym commicie nie wystarczy — należy unieważnić lub zmienić sekret, a w razie potrzeby również odpowiednio oczyścić historię.

## 15. Najważniejsze zasady

1. **Commit nie jest tym samym co push.** Commit zapisuje historię lokalnie; push wysyła commity na serwer.
2. **Pull nie jest tym samym co fetch.** Fetch pobiera informacje; pull dodatkowo próbuje zintegrować zmiany z aktualną gałęzią.
3. **Przed rozpoczęciem pracy na danym komputerze aktualizuj lokalną gałąź**, jeśli pracujesz ze wspólnym repozytorium.
4. **Używaj oddzielnych gałęzi do nowych funkcji.** Dzięki temu możesz rozwijać je niezależnie od głównej wersji projektu.
5. **Sprawdzaj `git status` przed commitem i przed synchronizacją.** To jedno z najbardziej użytecznych poleceń.

6. **Nie wysyłaj na serwer sekretów ani przypadkowych plików.** Korzystaj z `.gitignore`.
7. **Nie używaj `git push --force` bez zrozumienia skutków.** Współdzielone commity mogą zostać utracone lub odłączone od bieżącej historii.

Najprostszy zestaw poleceń, który warto opanować w pierwszej kolejności, to:

```
git status
git add .
git commit -m "Opis zmian"
git log --oneline
git switch -c feature/nazwa
git switch main
git merge feature/nazwa
git pull --ff-only
git push
```

Gdy te operacje staną się naturalne, Git przestanie być tylko narzędziem do robienia kopii projektu — stanie się sposobem kontrolowania całej historii jego rozwoju.